

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №1.

Разделение монолита на 2 микросервиса. Взаимодействие через HTTP

Цель занятия: освоить базовый подход к переходу от монолитного backend-приложения к микросервисной архитектуре на основе разделения системы на два самостоятельных сервиса и организации их взаимодействия по HTTP.

Задачи:

1. Изучить принцип разделения монолитного приложения на отдельные сервисы по зонам ответственности.
2. Научиться проектировать два независимых микросервиса на языке Go.
3. Освоить реализацию HTTP-взаимодействия между сервисами.
4. Научиться передавать и обрабатывать JSON-данные при межсервисном обмене.
5. Реализовать сервис, агрегирующий собственные данные и данные внешнего сервиса.
6. Отработать запуск, тестирование и проверку взаимодействия двух микросервисов.
7. Проанализировать преимущества и ограничения микросервисного подхода на учебном примере.

Теоретическая часть

1. Что такое монолит

Монолитное приложение — это система, в которой все основные функции собраны в одном процессе и обычно запускаются как один сервер. Например, в одном приложении могут одновременно находиться работа с пользователями, товарами, заказами, оплатой, логированием и административной панелью. Для небольших проектов это удобно: код находится в одном месте, запуск простой, отладка часто легче, а взаимодействие между частями системы происходит через прямые вызовы функций и методов.

Однако по мере роста проекта монолит начинает создавать трудности. Команда работает в одном большом кодовом дереве, изменение одной части может затронуть другую, сборка и деплой становятся тяжелее, а масштабировать приходится всё приложение целиком, даже если реальная нагрузка приходится только на один функциональный блок.

2. Что такое микросервис

Микросервис — это небольшой самостоятельный сервис, который отвечает за ограниченную бизнес-функцию и может быть запущен, обновлён и масштабирован отдельно от других частей системы. В микросервисной архитектуре одна система состоит не из одного большого приложения, а из набора сервисов, взаимодействующих по сети.

Например, один сервис может отвечать только за пользователей, другой — только за заказы, третий — только за уведомления. Каждый сервис может иметь свой HTTP API, собственные настройки, свои зависимости и даже свою базу данных.

3. Зачем разделять монолит

Разделение монолита на сервисы обычно делают не «потому что это модно», а когда в системе появляются реальные признаки роста:

разные части приложения развиваются с разной скоростью;

разным командам неудобно работать в одном коде;

нужно независимо выкатывать отдельные функции;

нагрузка распределена неравномерно;

одна часть системы должна быть особенно отказоустойчивой или масштабируемой.

Важно понимать, что микросервисы не всегда лучше монолита. Они дают гибкость, но усложняют инфраструктуру, тестирование, мониторинг, отладку и сетевое взаимодействие.

4. Как выделять микросервисы

Один из главных принципов — делить систему по ответственности. Не по техническим слоям вроде «контроллеры отдельно, модели отдельно», а по бизнес-областям. Например:

сервис пользователей;

сервис заказов;

сервис каталога;

сервис уведомлений.

В этой практической работе мы сделаем учебный пример из двух сервисов:

user-service — хранит и отдаёт информацию о пользователях;

order-service — хранит заказы и, при необходимости, обращается в user-service, чтобы получить данные о пользователе и вернуть агрегированный ответ.

Таким образом, order-service не хранит у себя полную информацию о пользователе, а получает её по HTTP.

5. HTTP-взаимодействие между сервисами

Когда один микросервис хочет получить данные у другого, он отправляет HTTP-запрос по URL. Например:

```
GET /users/1
```

В ответ другой сервис возвращает JSON:

```
{  
  "id": 1,  
  "name": "Иван Иванов",  
  "email": "ivan@example.com"  
}
```

Далее вызывающий сервис декодирует JSON, использует данные и формирует свой ответ.

Это очень простой и понятный способ организовать взаимодействие. Его достоинства:

- легко понять и протестировать;

- не нужны сложные брокеры и протоколы;

- можно проверять вручную через браузер, curl, Postman.

Но есть и проблемы:

- если один сервис недоступен, другой может работать частично или с ошибкой;

- возникают задержки сети;

- нужно обрабатывать таймауты, ошибки статусов, недоступность порта, битый JSON.

6. Что мы построим в этой работе

В данной практике будет реализован учебный мини-проект из двух сервисов.

Первый сервис — **user-service**. Он будет работать на порту 8081 и отдавать данные пользователей по маршруту:

```
GET /users/{id}
```

Второй сервис — **order-service**. Он будет работать на порту 8082 и отдавать:

```
GET /orders/{id} — просто заказ;
```

```
GET /orders/{id}/full — заказ вместе с данными пользователя.
```

Когда вызывается маршрут `/orders/{id}/full`, **order-service**:

находит заказ у себя;

берёт `user_id` из заказа;

обращается по HTTP в **user-service**;

получает информацию о пользователе;

собирает единый JSON-ответ.

Именно этот сценарий и демонстрирует переход от монолитной логики к сервисному взаимодействию.

Практическая часть

Общая идея проекта

В этой работе создаёте два отдельных приложения в рамках одной учебной папки:

```
user-service
order-service
```

Каждый сервис запускается отдельно, имеет свой `go.mod`, свой `main.go`, свои обработчики и свои данные в памяти.

Рекомендуемая структура каталогов

```
pzl-microservices/
  user-service/
    cmd/
      server/
        main.go
    internal/
      user/
        model.go
        repo.go
        handler.go
    go.mod

  order-service/
    cmd/
      server/
        main.go
    internal/
      order/
        model.go
        repo.go
        handler.go
        client.go
    go.mod
```

Шаг 1. Создание рабочей папки

Откройте PowerShell и перейдите в ту папку, где вы храните учебные проекты.

```
mkdir pz1-microservices
cd pz1-microservices
```

Шаг 2. Создание user-service

Создайте папки для первого сервиса:

```
mkdir user-service
cd user-service
go mod init example.com/user-service
mkdir cmd
mkdir cmd\server
mkdir internal
mkdir internal\user
```

Для macOS/Linux команды будут такими:

```
mkdir -p user-service/cmd/server
mkdir -p user-service/internal/user
cd user-service
go mod init example.com/user-service
```

Шаг 3. Модель пользователя

Создайте файл `internal/user/model.go`.

```
package user

type User struct {
    ID      int64  `json:"id"`
    Name    string `json:"name"`
    Email   string `json:"email"`
}
```

Шаг 4. Простое хранилище пользователей в памяти

Создайте файл `internal/user/repo.go`.

```
package user

import "errors"

var ErrUserNotFound = errors.New("user not found")

type Repo struct {
    data map[int64]User
}

func NewRepo() *Repo {
    return &Repo{
        data: map[int64]User{
            1: {ID: 1, Name: "Иван Иванов", Email:
"ivan@example.com"},
            2: {ID: 2, Name: "Мария Петрова", Email:
"maria@example.com"},
            3: {ID: 3, Name: "Алексей Сидоров", Email:
"alex@example.com"},
        },
    }
}

func (r *Repo) GetByID(id int64) (User, error) {
    u, ok := r.data[id]
    if !ok {
        return User{}, ErrUserNotFound
    }
    return u, nil
}
```

Шаг 5. HTTP-обработчик user-service

Создайте файл `internal/user/handler.go`.

```
package user

import (
    "encoding/json"
    "net/http"
    "strconv"
    "strings"
)

type Handler struct {
    repo *Repo
}

func NewHandler(repo *Repo) *Handler {
    return &Handler{repo: repo}
}

func (h *Handler) GetUserByID(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    // Ожидаемый путь: /users/{id}
    path := strings.TrimPrefix(r.URL.Path, "/users/")
    if path == "" || path == r.URL.Path {
        http.Error(w, "user id is required",
http.StatusBadRequest)
```

```

        return
    }

    id, err := strconv.ParseInt(path, 10, 64)
    if err != nil {
        http.Error(w, "invalid user id", http.StatusBadRequest)
        return
    }

    u, err := h.repo.GetByID(id)
    if err != nil {
        http.Error(w, "user not found", http.StatusNotFound)
        return
    }

    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    json.NewEncoder(w).Encode(u)
}

```

Шаг 6. Точка входа user-service

Создайте файл `cmd/server/main.go`.

```

package main

import (
    "log"
    "net/http"

    "example.com/user-service/internal/user"
)

func main() {
    repo := user.NewRepo()

```

```
handler := user.NewHandler(repo)

mux := http.NewServeMux()
mux.HandleFunc("/users/", handler.GetUserByID)

addr := ":8081"
log.Println("user-service started on", addr)

if err := http.ListenAndServe(addr, mux); err != nil {
    log.Fatal(err)
}
```

Шаг 7. Запуск и проверка user-service

Находясь в папке `user-service`, запустите сервис:

```
go run ./cmd/server
```

Ожидаемое сообщение в консоли:

```
user-service started on :8081
```

Откройте новое окно PowerShell и проверьте:

```
curl http://localhost:8081/users/1
```

Ожидаемый ответ:

```
{"id":1,"name":"Иван Иванов","email":"ivan@example.com"}
```

Проверьте несуществующего пользователя:

```
curl http://localhost:8081/users/999
```

Ожидаемо должен вернуться ответ со статусом 404.

Шаг 8. Создание order-service

Вернитесь в корневую папку проекта:

```
cd ..
mkdir order-service
cd order-service
```

```
go mod init example.com/order-service
mkdir cmd
mkdir cmd\server
mkdir internal
mkdir internal\order
```

Шаг 9. Модель заказа и агрегированный ответ

Создайте файл `internal/order/model.go`.

```
package order

type Order struct {
    ID      int64    `json:"id"`
    UserID  int64    `json:"user_id"`
    Item    string   `json:"item"`
    Price   float64  `json:"price"`
}

type UserDTO struct {
    ID      int64    `json:"id"`
    Name    string   `json:"name"`
    Email   string   `json:"email"`
}

type OrderWithUser struct {
    Order Order    `json:"order"`
    User  UserDTO  `json:"user"`
}
```

Шаг 10. Хранилище заказов в памяти

Создайте файл `internal/order/repo.go`.

```
package order
```

```

import "errors"

var ErrOrderNotFound = errors.New("order not found")

type Repo struct {
    data map[int64]Order
}

func NewRepo() *Repo {
    return &Repo{
        data: map[int64]Order{
            101: {ID: 101, UserID: 1, Item: "Ноутбук", Price:
79990},
            102: {ID: 102, UserID: 2, Item: "Мышь", Price:
2490},
            103: {ID: 103, UserID: 1, Item: "Клавиатура",
Price: 5990},
        },
    }
}

func (r *Repo) GetByID(id int64) (Order, error) {
    o, ok := r.data[id]
    if !ok {
        return Order{}, ErrOrderNotFound
    }
    return o, nil
}

```

Шаг 11. HTTP-клиент для вызова user-service

Создайте файл `internal/order/client.go`.

```
package order
```

```

import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

type UserServiceClient struct {
    baseURL string
    client  *http.Client
}

func NewUserServiceClient(baseURL string) *UserServiceClient
{
    return &UserServiceClient{
        baseURL: baseURL,
        client: &http.Client{
            Timeout: 3 * time.Second,
        },
    }
}

func (c *UserServiceClient) GetUserByID(id int64) (UserDTO,
error) {
    url := fmt.Sprintf("%s/users/%d", c.baseURL, id)

    resp, err := c.client.Get(url)
    if err != nil {
        return UserDTO{}, fmt.Errorf("request to user-service
failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode == http.StatusNotFound {

```

```

        return UserDTO{}, fmt.Errorf("user not found in user-
service")
    }

    if resp.StatusCode != http.StatusOK {
        return UserDTO{}, fmt.Errorf("user-service returned
status %d", resp.StatusCode)
    }

    var user UserDTO
    if err := json.NewDecoder(resp.Body).Decode(&user); err !=
nil {
        return UserDTO{}, fmt.Errorf("decode user response
failed: %w", err)
    }

    return user, nil
}

```

Шаг 12. Обработчики order-service

Создайте файл `internal/order/handler.go`.

```

package order

import (
    "encoding/json"
    "net/http"
    "strconv"
    "strings"
)

type Handler struct {
    repo    *Repo
    client *UserServiceClient
}

```

```

    }

    func NewHandler(repo *Repo, client *UserServiceClient)
    *Handler {
        return &Handler{
            repo: repo,
            client: client,
        }
    }

    func (h *Handler) GetOrderByID(w http.ResponseWriter, r
    *http.Request) {
        if r.Method != http.MethodGet {
            http.Error(w, "method not allowed",
            http.StatusMethodNotAllowed)
            return
        }

        path := strings.TrimPrefix(r.URL.Path, "/orders/")
        if path == "" || path == r.URL.Path {
            http.Error(w, "order id is required",
            http.StatusBadRequest)
            return
        }

        // Для /orders/{id}/full сюда заходить не нужно
        if strings.HasSuffix(path, "/full") {
            http.Error(w, "use /orders/{id}/full",
            http.StatusBadRequest)
            return
        }

        id, err := strconv.ParseInt(path, 10, 64)
        if err != nil {
            http.Error(w, "invalid order id", http.StatusBadRequest)

```

```

        return
    }

    order, err := h.repo.GetByID(id)
    if err != nil {
        http.Error(w, "order not found", http.StatusNotFound)
        return
    }

    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    json.NewEncoder(w).Encode(order)
}

func (h *Handler) GetOrderWithUser(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    // Ожидаемый путь: /orders/{id}/full
    path := strings.TrimPrefix(r.URL.Path, "/orders/")
    if !strings.HasSuffix(path, "/full") {
        http.Error(w, "bad route", http.StatusBadRequest)
        return
    }

    idPart := strings.TrimSuffix(path, "/full")
    idPart = strings.TrimSuffix(idPart, "/")

    id, err := strconv.ParseInt(idPart, 10, 64)
    if err != nil {
        http.Error(w, "invalid order id", http.StatusBadRequest)
    }
}

```

```

        return
    }

    order, err := h.repo.GetByID(id)
    if err != nil {
        http.Error(w, "order not found", http.StatusNotFound)
        return
    }

    user, err := h.client.GetUserByID(order.UserID)
    if err != nil {
        http.Error(w, "failed to get user data: "+err.Error(),
http.StatusBadGateway)
        return
    }

    result := OrderWithUser{
        Order: order,
        User:  user,
    }

    w.Header().Set("Content-Type",          "application/json;
charset=utf-8")
    json.NewEncoder(w).Encode(result)
}

```

Шаг 13. Точка входа order-service

Создайте файл cmd/server/main.go.

```

package main

import (
    "log"
    "net/http"

```

```

    "strings"

    "example.com/order-service/internal/order"
)

func main() {
    repo := order.NewRepo()
    client :=
order.NewUserServiceClient("http://localhost:8081")
    handler := order.NewHandler(repo, client)

    mux := http.NewServeMux()

    mux.HandleFunc("/orders/", func(w http.ResponseWriter, r
*http.Request) {
        if strings.HasSuffix(r.URL.Path, "/full") {
            handler.GetOrderWithUser(w, r)
            return
        }
        handler.GetOrderByID(w, r)
    })

    addr := ":8082"
    log.Println("order-service started on", addr)

    if err := http.ListenAndServe(addr, mux); err != nil {
        log.Fatal(err)
    }
}

```

Шаг 14. Запуск order-service

Откройте новое окно PowerShell, перейдите в папку order-service и ВЫПОЛНИТЕ:

```
go run ./cmd/server
```

Ожидаемое сообщение:

```
order-service started on :8082
```

Шаг 15. Проверка обычного маршрута заказа

Проверьте, что order-service отдаёт заказ без обращения к user-service:

```
curl http://localhost:8082/orders/101
```

Ожидаемый ответ:

```
{"id":101,"user_id":1,"item":"Ноутбук","price":79990}
```

Шаг 16. Проверка агрегированного маршрута

Теперь проверьте маршрут, который должен обращаться ко второму сервису:

```
curl http://localhost:8082/orders/101/full
```

Ожидаемый ответ:

```
{
  "order": {
    "id": 101,
    "user_id": 1,
    "item": "Ноутбук",
    "price": 79990
  },
  "user": {
    "id": 1,
    "name": "Иван Иванов",
    "email": "ivan@example.com"
  }
}
```

Это и есть главный результат практической работы: один сервис собрал ответ с использованием данных другого сервиса.

Шаг 17. Проверка сценария ошибки

Остановите `user-service` и снова вызовите:

```
curl http://localhost:8082/orders/101/full
```

Теперь `order-service` не сможет получить данные пользователя и должен вернуть ошибку уровня `502 Bad Gateway` или аналогичное сообщение о проблеме связи с внешним сервисом.

Это важный момент: в микросервисной архитектуре проблема одного сервиса влияет на работу другого. Именно поэтому в реальных системах важны таймауты, `retries`, `circuit breaker`, кэш и мониторинг.

Что получилось в итоге

Выполнив работу, студент построил простую распределённую систему из двух частей:

- `user-service` отвечает только за пользователей;
- `order-service` отвечает только за заказы;

для получения полной информации `order-service` обращается к `user-service` по HTTP.

С архитектурной точки зрения это уже не монолит, потому что данные и логика разделены между двумя независимыми процессами.

Что важно понять по итогам работы

В монолите `order`-модуль мог бы просто вызвать функцию `GetUserByID()` внутри того же приложения. Это было бы быстрее и

проще. Но в микросервисном варианте order-service больше не имеет прямого доступа к внутреннему коду user-service. Вместо этого он обращается к нему по сети, как к отдельному приложению.

Это даёт независимость сервисов, но добавляет новые обязанности разработчику:

- контролировать сетевые ошибки;
- следить за адресами сервисов;
- обрабатывать таймауты;
- проектировать API между сервисами;
- понимать, что отказ одного компонента может повлиять на другой.

Дополнительное задание

После того как базовый вариант заработал, можно предложить студентам одно или несколько усложнений.

Вариант 1. Добавить список заказов пользователя

Реализовать маршрут:

```
GET /orders/by-user/{userID}
```

который будет возвращать все заказы конкретного пользователя.

Вариант 2. Добавить маршрут списка пользователей

В user-service реализовать:

```
GET /users
```

с возвратом массива пользователей.

Вариант 3. Добавить логирование запросов

Подключить middleware логирования в каждом сервисе, чтобы в консоли выводились метод, путь и время выполнения запроса.

Вариант 4. Вынести адрес user-service в конфигурацию

Не хардкодить "http://localhost:8081", а читать адрес из переменной окружения.

Контрольные вопросы

1. Чем монолит отличается от микросервисной архитектуры?
2. Почему разделение системы на сервисы не всегда является лучшим решением?
3. Какую ответственность в этой работе несёт `user-service`, а какую — `order-service`?
4. Почему в микросервисной архитектуре важно обрабатывать таймауты и сетевые ошибки?
5. Что означает статус `502 Bad Gateway` в контексте взаимодействия сервисов?
6. Почему один сервис не должен напрямую обращаться к внутренним структурам кода другого сервиса?
7. Какие преимущества даёт разделение на сервисы при росте проекта?
8. Какие новые сложности появляются после такого разделения?

Подсказки и типичные ошибки

Если `order-service` не может обратиться к `user-service`, сначала проверьте, запущен ли первый сервис и совпадает ли порт.

Если в браузере или Postman вы видите ошибку 404, убедитесь, что маршрут введён полностью, например `/users/1`, а не просто `/users`.

Если `strconv.ParseInt` возвращает ошибку, значит в URL вместо числа передано что-то другое.

Если JSON не декодируется, проверьте теги `json:"..."` в структурах и убедитесь, что ответ действительно приходит в формате JSON.

Если один сервис работает, а другой нет, открывайте два отдельных терминала. Каждый сервис должен работать в своём процессе.